逆向与实现报告

掌瓦（掌上无畏契约）扫码登录完整方案

项目 Valorant-Plugin（TRSS-Yunzai / Yunzai V3 / ALemonJS）

逆向目标 掌上无畏契约 2.9.0 · com.tencent.apps.valorant

验证状态 微信 / QQ 两条链路均已真机扫码并取到战绩数据

0. 一句话结论

扫码登录能做，而且两条路都能做：微信零配置、纯 HTTP；QQ 需要一台浏览器。 拿到的凭证与抓包导入完全等价（cookie + scene），可以直接查战绩。

本文记录的是完整的推导过程、接口契约、踩过的坑和验证数据，不是"据说可以"。

1. 能力边界

	状态	说明
微信扫码登录	✅ 已验证	纯 HTTP，无需 appid / secret / 域名
QQ 扫码登录	✅ 已验证	需要 Chrome/Chromium，出码约 10 秒
换取掌瓦凭证	✅ 已验证	`login_by_qq` / `login_by_wechat` 都由掌瓦服务端完成 code→openid
查战绩	✅ 已验证	扫码凭证直接可用，与抓包导入同一套
凭证有效期	⚠️ 3 天	底层 web ticket 硬过期，到期需重扫或重抓
跨账号查询	❌ 不支持	掌瓦没有按昵称搜人的接口，只能查自己 / 同过场的人

2. 逆向对象与方法

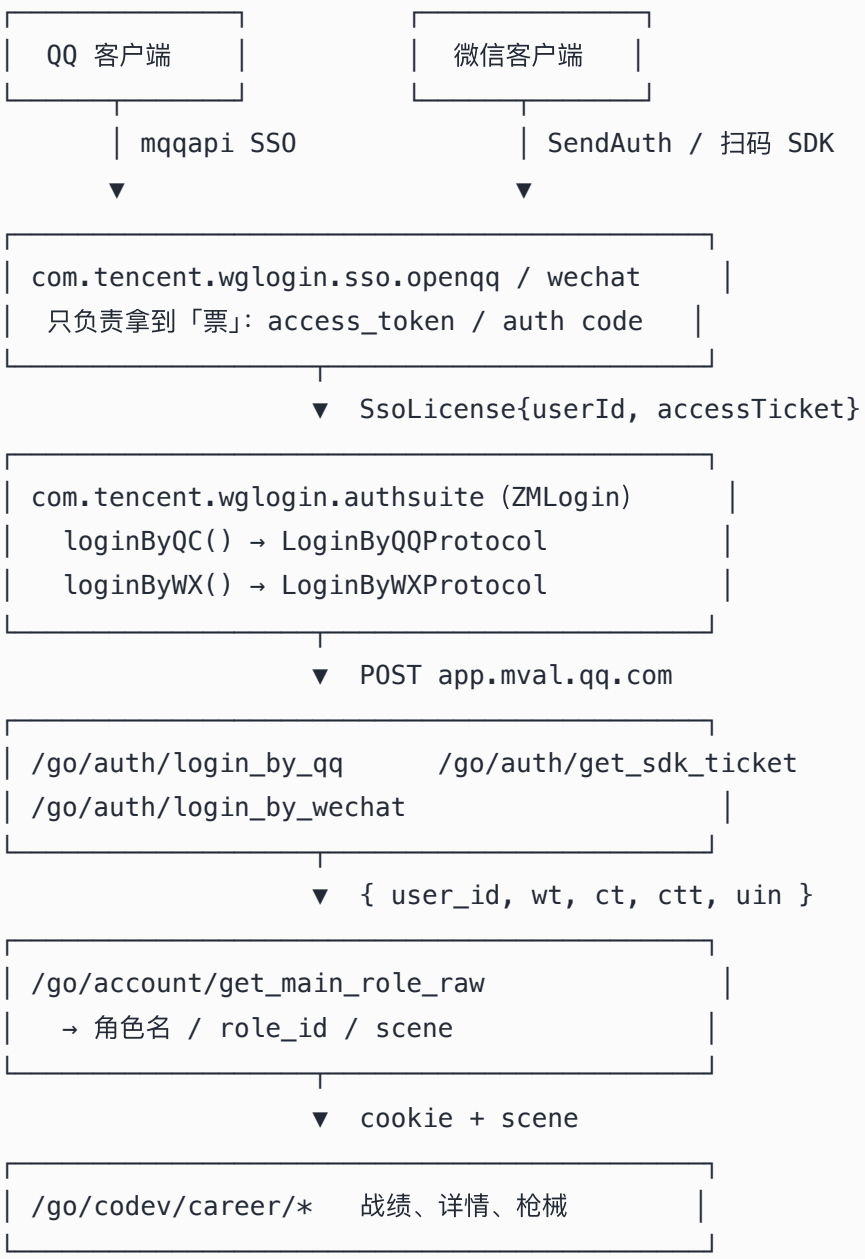
项	值
文件	`掌上无畏契约_2.9.0.APK`（111 MB）
包名	`com.tencent.apps.valorant`
架构	原生 Kotlin + React Native（`libhermes` / `libfabricjni`）+ 腾讯 QTL（掌盟）框架
DEX	17 个，约 5.75 万个类 / 37 万个方法
工具	jadx 1.5.6（单类反编译）+ 自写 DEX 解析器（`reapk/dexscan.py`，做字符串与交叉引用定向提取）

方法上的两个关键点：

1. **不要全量反编译**。5.75 万个类跑 jadx 要半小时以上，且大部分是广告/容器代码。 正确做法是先扫字符串池定位特征串，再用 DEX 交叉引用找到引用它的类，最后只反编译那几个类。
2. **Gson 默认用字段名当 JSON key**。所以只要从 DEX 里读到 `LoginByWXProtocol$Params` 的字段声明，就等于拿到了服务端的完整请求契约，不需要抓包也能写对请求体。

---

3. 登录架构全景



整件事能成立的前提，也是最容易看漏的一点：

`login_by_qq` / `login_by_wechat` 的请求体里**没有 secret**。 code→openid 的交换是**掌瓦服务端**用它自己的 AppSecret 做的。 所以我们不需要腾讯的密钥，只需要「一张票」——这正是扫码能替代抓包的原因。

4. 微信扫码：走的是「扫码 SDK」，不是「网站应用」

4.1 两套微信扫码，别搞混

这是本方案最早踩的坑，也是我一度错误判断"微信做不到"的原因：

	网站应用 `connect/qrconnect`	扫码 SDK `connect/sdk/qrconnect`
redirect_uri	**必须有，且要在开放平台备案**	**没有这个概念**
门槛	appid 归属 + 域名备案	一串 `sdk_ticket` + sha1 签名
code 从哪来	回调到你的服务器	长轮询接口直接吐给你
掌瓦能用吗	❌ 回调域是腾讯的	✅

手机 App 里那个「用微信扫一扫登录」用的就是后者。掌瓦复用它，我们原样复用掌瓦。

4.2 逆向来源

`classes8.dex` → `com.tencent.wglogin.authsuite.c`，方法 `a(Activity, a)`（内部叫 `loginWXByQRCode`）：

```
new GetWXServerTicketProtocol().post(activity, … {
    public void onSuccess(GetWXServerTicketResult r, boolean z) {
        String appid     = com.tencent.wglogin.authsuite.util.a.b;  // 运行时注
入
        String noncestr  = e.a(6);
        String timestamp = (System.currentTimeMillis() / 1000) + "";
        String origin = "appid=" + appid + "&noncestr=" + noncestr
                      + "&sdk_ticket=" + r.ticket + "&timestamp=" + timestamp;
        String sign = f.a(origin);                                  // sha1
hex
        DiffDevOAuthFactory.getDiffDevOAuth()
            .auth(appid, "snsapi_userinfo", noncestr, timestamp, sign,
listener);
    }
});
```

`DiffDevOAuth` 是微信官方 `libammsdk` 里的类，它内部做的事正好对应 DEX 里那两条 URL：

```
https://open.weixin.qq.com/connect/sdk/qrconnect?
appid=%s&noncestr=%s&timestamp=%s&scope=%s&signature=%s
https://long.open.weixin.qq.com/connect/l/qrconnect?f=json&uuid=%s
```

4.3 取票（这一步是整个方案的关键）

```
POST https://app.mval.qq.com/go/auth/get_sdk_ticket?
source_game_zone=agame&game_zone=agame
Content-Type: application/json
Cookie: clientType=9; openid=null; access_token=null

{"clienttype":9,"config_params":
{"client_dev_name":"23117RK66C","lang_type":0}}
```

免鉴权，直接返回：

```
{"data":{"ticket":"qeuRRdzufZaOpdhfU2Q4gGbgoxq-
NLbBLoPmh9nUqycSJTMoFHaQ4dN1XcR9TaOt…"},"result":0}
```

注意：必须是 **POST + JSON body**，GET 会返回 `404 page not found`。

4.4 签名

```
signature = sha1("appid={appid}&noncestr={noncestr}&sdk_ticket=
{ticket}&timestamp={timestamp}")
```

参数顺序不能变，这是微信的规则。`noncestr` 是随机串（App 里是 6 位，我们用 16 位 hex 也可以）。

4.5 出码

```
GET https://open.weixin.qq.com/connect/sdk/qrconnect
    ?appid=wxcbb49f1f39656c2a
    &noncestr={noncestr}
    &timestamp={timestamp}
    &scope=snsapi_userinfo
    &signature={signature}
```

成功返回（实测 68 700 字节）：

```
{
  "errcode": 0,
  "uuid": "061lJR7z2qTgnl2d",
  "appname": "掌上无畏契约",
  "qrcode": { "qrcodebase64": "/9j/4AAQSkZJRgABAQ…" }
}
```

`appname` 回显「掌上无畏契约」——这是**服务端替我们确认了 appid 归属正确**，比任何静态分析都硬。 图片是 **JPEG**（`/9j/` 开头，472×472），不是 PNG。

失败时 `errcode=40001` 表示签名不对或 ticket 过期。

4.6 轮询状态

```
GET https://long.open.weixin.qq.com/connect/l/qrconnect?f=json&uuid={uuid}
```

wx_errcode	含义	处理
408	未扫码	继续等
404	已扫，等待手机确认	提示用户点确认
405	已确认，**带** `wx_code`	拿 code 去换票
402	二维码过期	重新出码
403	用户取消	结束

⚠️ 只有 **405** 才带 code。404 只是"扫了但没点确认"，此时去换票必然失败。

4.7 换掌瓦凭证

```
POST https://app.mval.qq.com/go/auth/login_by_wechat?
source_game_zone=agame&game_zone=agame
Cookie: clientType=9; openid=null; access_token=null

{
  "clienttype": 9,
  "config_params": { "client_dev_name": "23117RK66C", "lang_type": 0 },
  "login_info": {
    "appid": "wxcbb49f1f39656c2a",
    "code": "<405 给的 wx_code>",
    "wx_info_type": 1,
    "check_third_type": 1
  },
  "mappid": 10200,
  "mcode": "132f0a77d34402abc8463d60100011d19b0e"
}
```

字段名来自 `LoginByWXProtocol$Params`（Gson 字段名 = JSON key）。
`check_third_type` 在 `wglogin.d#a(SsoLicense, f)` 里写死为 **1**。

错误码：喂假 code 会返回 `40029`（微信标准的 invalid code），这正好证明「交换发生在掌瓦服务端」，也证明我们的请求体格式完全正确。

4.8 实测记录

```
[1] ticket: {"data":{"ticket":"qeuRRdzufZaOpdhfU2Q4gGbgoxq-…"},"result":0}
[2] raw:    appid=wxcbb49f1f39656c2a&noncestr=11ce2b4a6121d8da&sdk_ticket=…
&timestamp=…
[3] HTTP 200 len 68700 → errcode=0, uuid=061lJR7z2qTgnl2d, appname=掌上无畏契约
[4] poll → {"wx_errcode":408, "wx_code":""}
```

出码耗时 **709 ~ 872 ms**。

---

5. QQ 扫码：页面式授权，绕开被黑洞的那一跳

5.1 我最初错在哪

第一版是纯 HTTP 手工追 `ptqrlogin → check_sig → auth://`，卡在 `check_sig` 上。那台机器的 DNS 确实被指到 `0.0.0.1`，四个公共 DNS 结果一致：

```
$ dig +short ptlogin4.openmobile.qq.com @223.5.5.5    → 0.0.0.1
$ dig +short ptlogin4.graph.qq.com      @119.29.29.29 → 0.0.0.1
$ dig +short ptlogin4.web.qq.com        @1.1.1.1      → 0.0.0.1
$ curl https://ptlogin4.openmobile.qq.com/            → 000（连不上）
```

我据此得出「换票前置机已下线，扫码做不到」——**错在把这条路当成了唯一的路。**

5.2 真正的落地页是 `m_authlist`

`m_authorize` 返回的不是二维码，而是一个跳转页，里面带着**服务端下发的** `h5sig`：

```
var src = "https://xui.ptlogin2.qq.com/cgi-bin/xlogin?appid=716027609
          &pt_3rd_aid=102061775&daid=381&…
&h5sig=MfBler9YgYCaIkB91wdEHEiRvJzgTqz6JTw7_14YvYM&loginty=7";
location.replace(src);
```

`loginty=7` + `h5sig` 走的是 openapi 版授权。扫码确认后，页面去的是：

```
GET https://openmobile.qq.com/oauth2.0/m_authlist?…&loginty=7
→ HTTP 200, 4246 字节
→ <p id="auth_tips">掌上无畏契约将获得以下权限:</p>
```

这台是活的。 授权在 `openmobile.qq.com` 域内完成，`check_sig` 是旧流程（`loginty=6`）的落点。走错了分支，才会撞上那三台死机器。

5.3 方案：让浏览器跑，我们只听

```
1  GET  openmobile.qq.com/oauth2.0/m_authorize?client_id=102061775

&scope=all&redirect_uri=auth://tauth.qq.com/&response_type=token&style=qr
2  页面 JS 自动跳转 → xui.ptlogin2.qq.com 渲染二维码
3  用户扫码 + 手机确认
4  页面内完成授权 → 跳 auth://tauth.qq.com/#access_token=…&pay_token=…
5  拦截这个回调，取 access_token
6  GET  graph.qq.com/oauth2.0/me?access_token=…&fmt=json  → openid
7  POST login_by_qq → 掌瓦凭证
```

三处信号同时监听（哪个先响算哪个，缺一不可）：

信号	命中场景
`page.on('request')`	`auth://` 协议导航
`page.on('framenavigated')`	导航不一定产生 request 事件
轮询 `location.href`	兜底，防上面两个都没触发

保险丝：一旦发现浏览器要去访问那三台黑洞主机，就地 `abort`， 把同一个 path/query 改投到活着的 `ptlogin4.qq.com` 集群 （实测：无效 sigx 会 302 到 `www.qq.com`，说明它认这套参数）。

5.4 出码细节（踩过的坑）

页面上的真码是：

```
<img id="qrimg" src="https://xui.ptlogin2.qq.com/ssl/ptqrshow?
s=8&e=0&appid=716027609&…">
```

但页面上还有一个**空的** `<img id="imgVerify">`。 第一版按 `img` 标签取第一张，抓到了那个空元素，截出一张 549 字节的空白图。

正确做法：选择器写 `#qrimg`，并且**回源取原图**—— 拿页面当前 cookie 去请求那个 `ptqrshow` 地址，得到 264×264 原始 PNG， 比截屏清晰、也不受页面缩放影响（同一个 `qrsig` 反复请求是幂等的，拿到的是同一张码）。

5.5 回调里没有 openid

`response_type=token` 的回调只有 `access_token` / `pay_token`，**没有 openid**。 要另外用标准接口换：

```
GET https://graph.qq.com/oauth2.0/me?access_token=…&fmt=json
→ {"openid":"…","client_id":"…"}
```

老版本会返回 JSONP `callback( {"openid":"…"} );`，需要兼容。

5.6 实测记录

```
[qq-page] 打开授权页：…/m_authorize?client_id=***&…&style=qr
✅ qq 出码成功（8898ms）  二维码 264×264 PNG
   poll#1: status=waiting msg=等待扫码
```

出码耗时 **8.9 ～ 9.9 秒**（要起浏览器 + 等页面 JS 加载）。

> **UA 很关键**：必须用手机 UA，末尾带 `tencent_game_emulator`（腾讯自己的模拟器标识）。 桌面 UA 会拿到空白页，出不了码。 另外浏览器要把 `navigator.webdriver` 抹掉。

---

6. 关键参数速查

项	值	来源
包名	`com.tencent.apps.valorant`	APK
QQ 互联 appid	`102061775`	`ShareInitTask`（`c.g()` 分支）
微信 appid（掌瓦）	`wxcbb49f1f39656c2a`	`ShareInitTask`
微信 appid（掌盟）	`wx5a4a8ac0fd48303a`	⚠️ 不是这个，用错必 40029
`mappid`	`10200`	`LoginByWXProtocol$Params` 默认
`mcode`	`132f0a77d34402abc8463d60100011d19b0e`	客户端指纹，随版本变
`clienttype`	`9`	Android 客户端
`pt_3rd_aid` / `daid`	`102061775` / `381`	QQ 登录页参数
`scope`（微信）	`snsapi_userinfo`	`auth(...)` 写死
`check_third_type`	`1`	`loginByWX` 写死
`device`	`23117RK66C`	小米机型，UA 与 `client_dev_name` 要一致

7. 工程实现

7.1 模块结构

```
bridge/login/
├── index.js       适配器注册表（qq / wx，可选 wxweb / mock）
├── session.js     会话状态机 + 资源释放（release）
├── qq.js          QQ 适配器：浏览器优先，纯 HTTP 兜底
├── qqpage.js      └ 页面式授权（真浏览器跑 m_authorize）
├── browser.js     └ 浏览器获取（puppeteer-core / puppeteer / 云崽宿主）
├── wechat.js      微信适配器：扫码 SDK，纯 HTTP
├── wxqr.js        └ sdk_ticket → 签名 → 出码 → 轮询
├── exchange.js    code / token → 掌瓦凭证（login_by_qq / login_by_wechat）
└── mock.js        假登录（仅 LOGIN_MOCK=1）
```

分层约定：插件只管交互（出图、轮询、提示），网关持有会话， provider 负责 `saveLogin()` 。加一个新平台 = 加一个 adap ter + 注册一行，插件不用改。

7.2 状态机

```
waiting ──扫码──> scanned ──确认──> confirmed ──commit──> 落库
   │                  │
   ├──超时──────────> expired
   ├──上游不认参数──> blocked
   └──用户取消──────> cancelled
```

`blocked` 是刻意设计的状态，不是异常补丁：上游不认这套请求时， 让用户干等到超时是最差的选择，连续 5 次空响应就承认失败并给出下一步建议。

7.3 凭证落库与多人隔离

```
bridge/mate.json                基础配置（接口路径、武器表）── 部署者一份
bridge/accounts/<user>.json     每个人的 cookie + scene ── 一人一份
```

`saveCredential()` 会**先落盘再验证**：立刻拿这份 cookie 打一次真实请求， 认出角色名与 `role_id` ；验证失败则删掉刚写的文件，不留半份坏凭证。

这条"绝不偷偷用 base 凭证"的规则很重要——否则每个群友都会看到机器人主人的战绩。

7.4 配置项

环境变量	作用	默认
`QQ_MODE=http`	强制 QQ 走纯 HTTP（排障对比用）	浏览器优先
`PUPPETEER_EXECUTABLE_PATH`	指定 Chrome 路径	自动探测
`LOGIN_TTL`	本地会话计时（秒）	180
`QQ_APPID` / `WX_APPID`	覆盖 appid	见速查表
`MVAL_MCODE` / `MVAL_DEVICE`	客户端指纹	APK 实测值

---

8. 端到端验证记录

8.1 全链路

```
STATUS waiting    :: 等待扫码
STATUS scanned    :: 已扫描，请在手机上点「确认登录」
STATUS confirmed  :: 登录成功：苦橘甘来
SAVED {"saved":true,"role":{"uid":"JA-10621b7…","name":"苦橘甘
来","roleId":"1383005380427"}}
```

扫码 → 确认 → 换票 → 认角色，**32 秒**完成。

8.2 落库结果

`bridge/accounts/demo.json`：

项	值
cookie	8 段（clientType / appid / acctype / openid / access_token / userId / accountType / tid）
scene	175 字符（v3_ 开头）
role	uid + name + roleId

8.3 用这份凭证查到的真实数据

地图	模式	特工	K/D/A	ACS	比分	结果
天枢云阙	竞技模式	炼狱	12/16/9	221	10-10	平
源工重镇	竞技模式	炼狱	9/8/4	304	8-2	胜
亚海悬城	竞技模式	芮娜	12/12/5	296	6-8	负

汇总：1 胜 1 负，胜率 50%，平均 ACS 274，K/D 1.45。

这一步的意义大于"登录成功"：它同时证明了三件事—— code 是真能用的（不是空壳）、掌瓦服务端完成了交换、凭证是完整可用的（不只是登录态）。

8.4 换环境复验：Docker 里的 TRSS-Yunzai

同一份代码搬到容器里（`TRSS_AllBot`，Arch Linux，Node 26，Chromium 154）重新跑了一遍， 结论是**逻辑零改动、只需要补环境**：

环节	结果
加载登录模块	`adapters: qq/wx`
微信出码（直连适配器）	**937 ms**
微信出码（走网关 HTTP）	**1.0–2.8 s**
扫码 → 确认	**3 s**（`23:24:13` → `23:24:16`）
落库	`saved:true`，8 段 cookie + 175 字符 scene，`source: scan:wx`
用新凭证查战绩	3 场：天枢云阙 12-16-9 / 源工重镇 9-8-4 / 亚海悬城 12-12-5，汇总 1 胜 1 负、ACS 274、K/D 1.45
QQ 出码（要浏览器）	**7258 ms**，398×398 真二维码

容器里额外踩到、本地完全不会遇到的问题（缺 CJK 字体导致出图豆腐块、`/dev/shm` 只有 64MB、 网关不重启就一直跑老适配器）单独整理成了 `Docker云崽部署与扫码登录.md`。

9. 失效点与维护手册

按"多久会坏"排序：

项	风险	坏了怎么办
`jsver`	腾讯每几周换一次	QQ 纯 HTTP 版从 xlogin 页面现抓；浏览器版不需要
`mcode` / `device`	随客户端版本变	重新从 APK 提，或抓包看 UA
`signature` / `h5sig`	一次性签名	浏览器版由页面自己生成；微信版每次现算
`sdk_ticket`	有有效期（分钟级）	每次出码都重新取一次，不要缓存
`scene`	会话级 blob，与"哪一页"绑定	列表/概览用一个（175 字符），单局详情用另一个（91 字符）
凭证	硬 3 天过期	重扫一次（微信 30 秒），或重新抓包

排障入口：

```
node tools/test-login.mjs wx                  # 微信：出码 + 状态机
node tools/test-login.mjs wx --save --query   # 微信：落库 + 查最近 3 场
node tools/test-login.mjs qq                  # QQ：约 10 秒出码
node tools/test-login.mjs qq --http           # 强制纯 HTTP，对比排障
node bridge/providers/mateapp-cli.mjs         # 打印战绩接口原始 JSON
```

10. 踩坑清单（按踩的顺序）

#	现象	真因	修法
1	`ptqrlogin` 稳定返回 200 + 空 body	用了网页站的参数（`daid=383`、`pt_3rd_aid=0`、浏览器 UA）	换成掌瓦 App 那套：`daid=381`、`pt_3rd_aid=102061775`、`mval` UA + `X-Requested-With`
2	二维码扫不动	`ptqrshow` 不传 `s` 只有 99×99	`s=8` → 264×264
3	轮询被判"非本客户端"	`jsver` 硬编码且已过期	从 xlogin 页面现抓
4	扫码成功但拿不到 token	跟了 `check_sig`（死主机），而新流程走 `m_authlist`	改用浏览器跑页面，只监听回调
5	二维码是 549 字节空白	选中了空的 `#imgVerify`	选择器改 `#qrimg` + 回源取原图
6	微信扫码"设计上不可行"	只看到要白名单的 `qrconnect`	改用扫码 SDK `sdk/qrconnect`
7	换票必 40029	appid 用成了掌盟的 `wx5a4a8ac0fd48303a`	掌瓦是 `wxcbb49f1f39656c2a`
8	报"查不了「[object Object]」"	`getMatches({account})` 要字符串，传了对象	传游戏名（或空串表示自己）
9	战绩显示 0 场	`getMatches` 返回 `{list, summary}`，不是数组	取 `.list`

11. 合规提醒

这套东西只用于**查你自己账号**的战绩。

- 抓包 / 复用登录态可能违反腾讯的用户协议，账号风险自负；
- 不要把他人的凭证、cookie 存进仓库或分发给第三方；

- `bridge/mate.json`、`bridge/accounts/`、`bridge/peers.json` 都已在 `.gitignore`，保持这样。

附录 A：完整请求清单

用途	方法	地址
取 sdk_ticket	POST	`app.mval.qq.com/go/auth/get_sdk_ticket`
微信出码	GET	`open.weixin.qq.com/connect/sdk/qrconnect`
微信轮询	GET	`long.open.weixin.qq.com/connect/l/qrconnect`
微信换票	POST	`app.mval.qq.com/go/auth/login_by_wechat`
QQ 授权页	GET	`openmobile.qq.com/oauth2.0/m_authorize`
QQ 授权确认	GET	`openmobile.qq.com/oauth2.0/m_authlist`
QQ 换 openid	GET	`graph.qq.com/oauth2.0/me`
QQ 换票	POST	`app.mval.qq.com/go/auth/login_by_qq`
角色	POST	`app.mval.qq.com/go/account/get_main_role_raw`
战绩列表	POST	`app.mval.qq.com/go/codev/career/record/list`

附录 B：相关文件

文件	内容
`docs/掌瓦扫码登录完整方案.md`	本文
`docs/掌瓦登录链路逆向.md`	逆向过程与证据（更细的类/字段级记录）
`docs/Docker云崽部署与扫码登录.md`	容器（TRSS-Yunzai）里的部署实录与排障
`bridge/login/README.md`	登录模块的实现说明与接口
`bridge/providers/mateapp.md`	战绩接口与字段口径
`bridge/providers/mateapp-scanlogin.md`	早期排查存档（部分结论已被本文推翻）

生成自 docs/掌瓦扫码登录完整方案.md